

Documentação Projeto ETL Airflow

Objetivo: Construir uma pipeline ETL buscando os dados do clima de São Paulo da api OpenWeather para construir uma tabela.

Tecnologias Usadas:

Python - ETL

Pandas - Apenas pra normalização dos dados

Apache Airflow - para orquestração

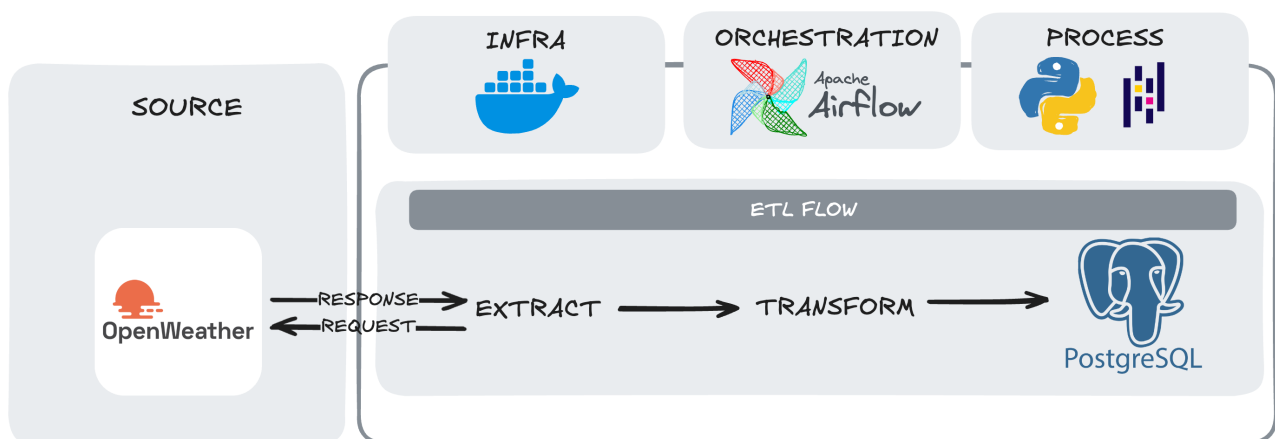
PostgreSQL – armazenamento

Wsl (linux) – linha de comandos e uso do docker

Docker – criação e persistencia de volume, containers do PostgreSQL e Airflow

Arquitetura:

Esta arquitetura realiza um pipeline ETL onde os dados são coletados da API do OpenWeather, processados com Python e armazenados no PostgreSQL. O Apache Airflow gerencia e automatiza o fluxo, enquanto o Docker mantém os serviços organizados em containers.



1 – Extract: Extrair dados da api OpenWeather com python (**extract_data.py**)

```
You, 2 days ago | 1 author (You)
1  import requests
2  import json
3  from pathlib import Path
4
5
6  def extract_weather_data(url:str) -> list:
7      try:
8          response = requests.get(url)
9          response.raise_for_status()
10
11          data = response.json()
12
13          output_path = 'data/weather_data.json'
14          output_dir = Path(output_path).parent
15          output_dir.mkdir(parents=True, exist_ok=True)
16
17          with open(output_path, 'w') as f:
18              json.dump(data, f, indent=4)
19
20          return data
21
22      except requests.exceptions.RequestException as e:
23          print(f"Erro na requisição: {e}")
```

2 – normalização dos dados ‘weather_data.json’ feito com pandas no jupyter notebook

```
path = '../data/weather_data.json'
import pandas as pd
import json

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

with open(path) as f:
    data = json.load(f)

df = pd.json_normalize(data)
df.head()
```

	weather	base	visibility	dt	timezone	id	name	cod	coord.lon	coord.lat	...	main.grnd_level	wind.speed	wind.deg	wind.gust	cloud
0	['id': 804, 'main': 'Clouds', 'description': 'stations']	10000	1781292639	-10800	3448439	São Paulo	200	-46.6361	-23.5475	...	923	1.79	318	2.68		

3 – Transform: Aqui foi feito a limpeza, padronização dos dados e colunas (**transform_data.py**)

- Foram excluídas 3 colunas:

```
['weather', 'weather_icon', 'sys.type']
```

- O horário dessas colunas foi ajustado e convertido para o horário de SP

```
['datetime', 'sunrise', 'sunset']
```

- E todas as colunas da variável **columns_to_rename** foram renomeadas

```
7 columns_to_drop = ['weather', 'weather_icon', 'sys.type']
8 columns_to_normalize_datetime = ['datetime', 'sunrise', 'sunset']
9 > columns_to_rename = {          You, 2 days ago • pipeline ETL concluída ...
38
39
40 > def create_dataframe(path_name:str) -> pd.DataFrame: ...
53
54 > def normalize_weather_columns(df: pd.DataFrame) -> pd.DataFrame: ...
67
68 > def drop_columns(df: pd.DataFrame, columns_names:list[str]) -> pd.DataFrame: ...
73
74 > def rename_columns(df: pd.DataFrame, columns_names:dict[str,str]) -> pd.DataFrame: ...
79
80 def normalize_datetime_columns(df: pd.DataFrame, columns_names:list[str]) -> pd.DataFrame:
81     print(f"\n Convertendo colunas para datetime: {columns_names}")
82     for name in columns_names:
83         df[name] = pd.to_datetime(df[name], unit='s', utc=True).dt.tz_convert('America/Sao_Paulo')
84     print(f"\nColunas convertidas para datetime")
85     return df
86
87 def data_transformation():
88     print("\n Iniciando transformações")
89     df = create_dataframe(path_name)
90     df = normalize_weather_columns(df)
91     df = drop_columns(df, columns_to_drop)
92     df = rename_columns(df, columns_to_rename)
93     df = normalize_datetime_columns(df, columns_to_normalize_datetime)
94     print("\n Transformações concluídas")
95     return df
```

4 – Load: Nessa parte os dados foram carregados após a limpeza onde conectamos com o PostgreSQL e guardamos a tabela em sql (**load_data.py**)

```
You, yesterday | 1 author (You)
1  from sqlalchemy import create_engine, text
2  from urllib.parse import quote_plus
3  from dotenv import load_dotenv
4  from pathlib import Path
5  from os import getenv
6  import pandas as pd
7
8  env_path = Path(__file__).resolve().parent.parent / 'config' / '.env'
9  load_dotenv(env_path)
10
11 user = getenv('user')
12 password = getenv('password')
13 database = getenv('database')
14 host = 'host.docker.internal'
15 # host = 'localhost'
16
17 def get_engine():
18     print('conectando com o banco...')
19     return create_engine(
20         f"postgresql+psycopg2://{user}:{quote_plus(password)}@{host}:5432/{database}"
21     )
22
23 engine = get_engine()
24
25 def load_weather_data(table_name:str, df):
26     df.to_sql(
27         name=table_name,
28         con=engine,
29         if_exists='append',
30         index=False
31     )
32
33     print('dados carregados com sucesso!\n')
34
35     df_check = pd.read_sql(f"SELECT * FROM {table_name}", con=engine)
36     print(f"total de registros na tabela: {len(df_check)}\n")
37
```

5 – DAG: Uma **DAG (Directed Acyclic Graph)** no Airflow é um fluxo de tarefas organizado em etapas, que define a ordem de execução de um processo. Ela permite agendar, controlar dependências e monitorar pipelines de dados, garantindo que cada tarefa seja executada no momento correto. Então usei as funções antes definidas e orquestréi a ordem em que deveriam ocorrer

```
7 path.insert(0, '/opt/airflow/src')
8
9 from extract_data import extract_weather_data
10 from transform_data import data_transformation
11 from load_data import load_weather_data
12 from dotenv import load_dotenv
13
14 env_path = Path(__file__).resolve().parent.parent / 'config' / '.env'
15 load_dotenv(env_path)
16
17 API_KEY = getenv('api_key')
18 url = f'https://api.openweathermap.org/data/2.5/weather?q=Sao Paulo,BR&units=metric&appid={API_KEY}'
19
20 @dag(
21     dag_id='weather_pipeline',
22     default_args={
23         'owner': 'airflow',
24         'depends_on_past': False,
25         'retries': 2,
26         'retry_delay': timedelta(minutes=5)
27     },
28     description='Pipeline ETL - Clima SP',
29     schedule='0 */1 * * * ',
30     start_date=datetime(2026, 6, 17),
31     catchup=False,
32     tags=['weather', 'etl']
33 )
34
35 def weather_pipeline():
36
37     @task
38     def extract():
39         extract_weather_data(url)
40
41     @task
42     def transform():
43         df = data_transformation()
44         df.to_parquet('/opt/airflow/data/temp_data.parquet', index=False)
45
46     @task
47     def load():
48         import pandas as pd
49         df = pd.read_parquet('/opt/airflow/data/temp_data.parquet')
50         load_weather_data('sp_weather', df)
51
52     extract() >> transform() >> load()
53
54 weather_pipeline()
```

Com tudo pronto inicializo o Docker fiz pelo terminal mas pode fazer pelo software do docker

	Na...	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
☐	☒	pipeline_weathr	-	-	60.52%	3.7GB / 54.12GB	47.93%	272.6MB / 347.3i	51.61MB / 54.07i	20, 5, 0, 4, 9, 4, 1	6 minutes a	
☐	☒	postgres-1	e2c81ad1ec05	postgres:1	6.8%	90.93MB / 7.73G	1.15%	74.8MB / 15.3ME	25MB / 26.6MB	20	7 minutes a	
☐	☒	redis-1	9a0679661722	redis:7.2-b	0.09%	10.63MB / 7.73G	0.13%	34.3MB / 0B	353KB / 271KB	5	7 minutes a	
☐	☐	airflow-init-1	3d855b5b12f5	apache/air	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	7 minutes a	
☐	☒	airflow-schec	c5be9ec40d9e	apache/air	0.72%	386.4MB / 7.73G	4.88%	12MB / 52.5MB	5.4MB / 5.99MB	4	6 minutes a	
☐	☒	airflow-trigge	c2931cbb9f9e	apache/air	1.32%	392.7MB / 7.73G	4.96%	18.9MB / 49.6ME	1.05MB / 1.5MB	9	6 minutes a	
☐	☒	airflow-dag-p	e882b5d19606	apache/air	51.39%	454.8MB / 7.73G	5.75%	102MB / 98.5MB	18.8MB / 16.9ME	4	6 minutes a	
☐	☒	airflow-apise	3947326f7da2	apache/air 8080:8080	0.07%	295.7MB / 7.73G	3.74%	16.5MB / 51.3ME	719KB / 2.44MB	13	6 minutes a	
☐	☒	airflow-workr	8b7a69c3dc26	apache/air	0.13%	2.11GB / 7.73GB	27.32%	14.1MB / 80.1ME	328KB / 385KB	24	6 minutes a	

encontro a weather_pipeline

weather_pipeline etl, weather		
Schedule	Latest Run	Next Run
0 */1 ***	2026-06-20 19:00:00	2026-06-20 20:00:00

funciona perfeitamente

<

visualizando a tabela no pgAdmin do PostgreSQL ficou mais ou menos assim:

Query Query History

1

SELECT *

2

FROM sp_weather

Data Output Messages Notifications